

Continual-Learning GNN Intrusion Detection System

Research-paper reference — tech stack, methodology, hyper-parameters, results and reproducibility.
Author: Samarth. Every number is produced by the repository code.

1. Abstract (adapt as needed)

We present a network intrusion detection system that models each window of network flows as a graph (hosts as nodes, flows as edges with 83 CICFlowMeter features) and classifies edges with an **E-GraphSAGE** graph neural network. To learn new attack categories over time **without catastrophic forgetting**, we combine **Elastic Weight Consolidation (EWC)** with **subgraph replay**. On the error-corrected CIC-IDS2017 and CSE-CIC-IDS2018 datasets the graph model reaches macro-F1 0.915–0.964 while retaining 0.998–1.000 recall on the first attack after all later tasks, and detects some never-trained attacks (unseen DoS at 98.6% where a per-flow model sees <0.1%). We add a product layer (incident grouping, novelty/open-set flagging, conformal abstention, calibration, dry-run response) and an online teach-and-adapt deployment path. We are explicit about limitations: the graph advantage depends on attacks coming from few hosts, and a lab-trained model does not generalise to live traffic until taught.

2. Technology stack

Language & machine learning

Language	Python 3.12.10
Deep learning	PyTorch 2.14.0 (CUDA 12.6)
Graph neural network	PyTorch Geometric 2.8.0 (no compiled extensions)
Gradient boosting	XGBoost 3.4.1 (static baseline)
Classic ML / metrics	scikit-learn 1.9.1
Streaming / drift	river 0.26.1 (ADWIN)
Numerics / data	NumPy 2.5.3, pandas 3.0.5, SciPy 1.18.1, PyArrow 25.0.1
Plots	matplotlib 3.11.2

Backend, API & data

Web framework	FastAPI 0.141.1 + Uvicorn 0.53.0
ORM / database	SQLAlchemy 2.0 · SQLite (local) / PostgreSQL + TimescaleDB
HTTP client	httpx 0.28.1
Config	PyYAML 6.0.3
Flow extraction	CICFlowMeter (GintsEngelen fork, pinned, in Docker)
Packet capture	Wireshark / dumpcap + Npcap (sensor)

Frontend & tooling

Dashboard	Plain ES-module JavaScript (no framework / bundler), Chart.js, d3-force
Desktop app	Python tkinter (control center)
Containers	Docker (flow meter + isolated cyber range)
Testing	pytest 9.1.1 (103 tests), Playwright 1.63 (UI smoke)

Hardware for reported numbers: NVIDIA GeForce GTX 1650 (4 GB), 8-core CPU, 24 GB RAM, Windows 11.

3. System architecture

Five layers, run as local processes (scripts/run_stack.ps1):

Data	SQLite (local) or PostgreSQL + TimescaleDB
Ingestion	data preparation + database seed (one-shot)
ML service :8001	models, cached graphs, live scoring, demo stream
Public API :8000	REST — results, predict, incidents, live traffic
Dashboard :8080	the web console

Live-traffic path: sensor (capture/replay) → CICFlowMeter (pinned, Docker) → flows → ML service scores → console per site.

Deployment is passive (SPAN port / flow exporter), dry-run only.

4. Datasets

Dataset	Release	Tasks (order)	Flows
CIC-IDS2017	error-corrected (Engelen 2021)	BruteForce → DoS → WebAttack → Infiltration → Botnet → PortScan → DDoS (7)	all
CSE-CIC-IDS2 018	error-corrected (Liu 2022)	BruteForce → DoS → DDoS → WebAttack → Infiltration → Botnet (6)	15% sample

Both are laboratory captures with block-scheduled attacks; 83 CICFlowMeter features per flow.

5. Method & model architecture

Graph construction

- Flows sorted by time, cut into **windows of 5,000 flows** (never crossing a task boundary).
- Per window: unique IPs → nodes; each flow → a directed edge carrying its 83 scaled features.
- Node features: degree-based (in / out / total). IP addresses define the graph, never used as features.
- Scaling: signed log1p → StandardScaler (fit on task-1 train only) → clip to ± 10 .

E-GraphSAGE edge classifier

Hidden dimension	64
Layers	2
Dropout	0.2
Edge skip	classifier also sees the edge's own projected features
Parameters	69,128 (per-flow FFNN baseline: 55,432)

Continual learning

- **EWC** (online): protects weights important to earlier tasks; Fisher from 50 batches; λ , γ chosen on validation.
- **Subgraph replay**: reservoir pool of 10 windows per attack category; 2 replayed per training step (fixed budget).
- Loss = $L(\text{new}) + \text{replay_weight} \cdot L(\text{replay}) + \text{EWC penalty}$.

6. Hyper-parameters (exact values used)

Optimizer	Adam	EWC λ (selected)	10 (val; default 100)
Learning rate	0.001	EWC γ (selected)	0.9 (val; default 1.0)
Weight decay	0.0	Fisher batches	50
Grad clip	5.0	Replay graphs / class	10
Epochs (GNN / FFNN)	10 / 10 per task	Replay graphs / step	2
Class-weight power / clip	0.5 / [0.05, 50]	Window size	5,000 flows
Seeds	42, 43, 44 (temporal: 5)	ADWIN δ	0.002

7. Evaluation protocol & metrics

- **Splits:** interleaved (optimistic), temporal (train on earlier traffic of each attack, test on latest), leave-one-attack-out (unseen attacks), IP-remap (permute / randomise sources).
- **Metrics:** macro-F1 over seen tasks, per-task recall, retention (recall on task 1 after all later tasks), backward transfer (BWT), false-positive rate, detection rate; ECE/MCE/Brier (calibration); AUROC/TPR (open-set); incident counts; serving latency / throughput.
- **Statistics:** paired bootstrap confidence intervals and Wilcoxon signed-rank over seeds.

8. Key results

Result	Value
Macro-F1 (2017, temporal, 5 seeds)	0.915 $\pm$ 0.031 (95% CI 0.888–0.935)
Macro-F1 (2017, interleaved, 3 seeds)	0.964 $\pm$ 0.020
Macro-F1 (2018, 3 seeds)	0.911 $\pm$ 0.042
Graph vs per-flow (temporal / interleaved)	+0.044 / +0.036 macro-F1
Retention of first attack	0.998–1.000 (naive retraining: 0)
Unseen DoS (leave-one-out, 2018, 3 seeds)	98.6% (per-flow FFNN $\leq$ 0.07%)
Topology dependence (random sources, 2017)	0.952 $\rightarrow$ 0.431; FFNN unaffected
Serving speed (GTX 1650)	5.5 ms / 5,000-flow window ($\approx$ 630k flows/s)

9. Contributions / novelty

- 1 EWC + subgraph replay applied to **graph-based** flow IDS for class-incremental attack learning, with the three standard EWC traps handled correctly.
- 2 Evidence that **graph structure** detects some never-seen attacks a per-flow model cannot.
- 3 A product + deployment layer with **online teach-and-adapt** (gated against forgetting), novelty flagging, conformal abstention and dry-run response.
- 4 An honest characterisation of **when the approach fails** (topology dependence; lab-to-live gap).

10. Related work (reviewed)

40 papers reviewed (full list in docs/GNN-IDS_Project_Guide.pdf §5). Key anchors: E-GraphSAGE (Lo et al. 2022); EWC (Kirkpatrick et al. 2017); experience replay for continual learning; ADWIN (Bifet & Gavalda 2007); dataset-error corrections (Engelen 2021, Liu 2022, Lanvin 2022); GNN-for-IDS survey (Bilot et al. 2023); continual learning for NIDS (Amalapuram 2022).

11. Limitations (state these — they strengthen the paper)

- Never blocks traffic — dry-run proposals only.
- Graph advantage assumes attackers are few hosts; NAT / spoofing / distributed sources break it.
- A real modern attack (e.g. live nmap) is not detected out of the box — lab-to-live distribution gap.
- Drift detection needs delayed labels; unseen-attack detection is unreliable for some categories.
- Two lab datasets with block-scheduled attacks; not validated on production traffic.

12. Reproducibility

One command regenerates every number from the raw dataset zips:

Command	<code>python -m experiments.reproduce_all --dataset cicids2017</code>
Provenance	every results folder has <code>run_info.json</code> (seed, config, versions, GPU, command, data hash)
Tests	103 automated tests; UI smoke test over all console pages

13. How to cite the datasets

CIC-IDS2017 / CSE-CIC-IDS2018: Sharafaldin et al., ICISSP 2018. Error-corrected releases: Engelen et al. (WTMC 2021) and Liu et al. (IEEE CNS 2022). Cite the corrected releases, since all results use them.

For prose, figures and the full 40-paper review, see docs/GNN-IDS_Project_Guide.pdf and docs/GNN-IDS_Technical_Documentation.pdf.